

Key-Value Store v4

Heartbeat, failover e split brain

Architetture dei Sistemi Distribuiti

Lezione 13

Dopo la replica primary-secondary resta una domanda inevitabile:

chi decide se il primary esiste ancora?

Se non rispondiamo a questa domanda:

- il sistema puo' restare bloccato su un leader morto;
- oppure puo' promuovere troppo presto un follower;
- in entrambi i casi il contratto di scrittura diventa ambiguo.

Questa tappa introduce:

- heartbeat periodici dal leader;
- timeout locali di sospetto;
- promozione del follower;
- rischio di split brain come problema osservabile.

Non siamo ancora a un vero protocollo di consenso. Siamo nel punto in cui il sistema prova a restare vivo con strumenti ancora fragili.

Meccanismo minimale:

- il primary invia heartbeat periodici;
- il secondary aggiorna il proprio ultimo istante osservato;
- se il tempo supera una soglia, il secondary sospetta un guasto.

Heartbeat e timeout sono meccanismi semplici, ma il loro significato va trattato con molta cura.

Timeout Non E' Verita'

Un timeout significa solo:

- non ho visto heartbeat abbastanza in fretta.

Non significa automaticamente:

- il leader e' certamente crashato;
- il leader e' isolato dal mondo;
- nessun altro nodo lo vede piu'.

Questa distinzione e' il cuore teorico della lezione.

```
if role == "secondary" and now - last_heartbeat > timeout:  
    term += 1  
    role = "primary"  
    leader_id = self_id
```

La promozione e' una transizione locale osservabile:

- migliora la liveness se il leader e' davvero morto;
- puo' essere prematura se il timeout nasce da ritardo o partizione.

Una vista informale del follower:

- `SecondaryStable -> ReceiveHeartbeat -> SecondaryStable`
- `SecondaryStable -> NoHeartbeatForTooLong -> Promote`

L'interesse qui non e' il formalismo in se'. E' la possibilita' di chiedersi:

- quali transizioni sono lecite;
- quali sono solo sospetti;
- quali conseguenze hanno sul contratto delle scritture.

Caso favorevole:

- ➊ A e' primary e crasha davvero;
- ➋ B smette di ricevere heartbeat;
- ➌ il timeout scade;
- ➍ B si promuove;
- ➎ il sistema torna ad avere un nodo scrivibile.

Dal punto di vista della liveness, questo e' un successo.

Caso pericoloso:

- ➊ A continua a vivere ma smette di farsi percepire da B;
- ➋ B non riceve heartbeat e si promuove;
- ➌ A resta convinto di essere primary;
- ➍ entrambi accettano scritture.

Ora il sistema non ha più una nozione univoca di autorità di scrittura.

Perche' Questo E' Un Problema Di Safety

Lo split brain non e' solo “confusione sui ruoli”. E' una violazione di safety perche':

- due storie di update incompatibili possono crescere in parallelo;
- il sistema non sa piu' dire quale sia lo stato canonico;
- la convergenza futura richiedera' scelte arbitrarie o perdita di informazione.

```
STATUS
```

```
OK node=A role=primary term=1 leader=A
```

Il laboratorio rende visibili:

- ruolo corrente;
- termine locale;
- leader percepito.

Questo aiuta a trasformare il failover da intuizione a fenomeno osservabile e discutibile.

Liveness:

- il timeout evita di attendere per sempre un leader assente;
- il follower puo' fare progresso.

Safety:

- un timeout troppo aggressivo puo' creare due leader;
- una promozione prematura puo' generare divergenza.

La stessa soglia che salva la continuita' puo' indebolire l'unicita' della leadership.

Perche' Due Nodi Non Bastano

Con due nodi manca una maggioranza indipendente.

- ogni nodo ha solo la propria vista locale;
- in caso di ambiguita' non esiste un terzo giudice;
- nessuno puo' distinguere in modo robusto crash da partizione.

Per questo heartbeat e timeout bastano a mostrare il problema, ma non a risolverlo bene.

Failover pulito:

- ➊ avviare A come primary e B come secondary;
- ➋ osservare STATUS;
- ➌ eseguire CRASH su A;
- ➍ attendere il timeout;
- ➎ verificare la promozione di B.

Split brain volontario:

- ➊ avviare A e B;
- ➋ sospendere gli heartbeat;
- ➌ attendere la promozione del follower;
- ➍ osservare il momento in cui entrambi si credono scrivibili;
- ➎ discutere che cosa significhi OK in questa fase.

Per uscire davvero da questo livello di fragilita' servono:

- piu' nodi;
- decisioni basate su cardinalita' e intersezione;
- regole piu' forti del semplice sospetto locale.

Questa e' la motivazione naturale per la tappa sui quorum.

Heartbeat e timeout migliorano il failover. Ma non bastano a difendere l'unicita' del leader.

- il sistema puo' restare vivo;
- e proprio per questo puo' diventare semanticamente ambiguo;
- la continuita' del servizio non coincide con la correttezza del ruolo.